Editorial Copilot — structured briefing drafts from audited data

SPEC-SW-011 v0.1 DRAFT Jeremie Nunez

1 Context

The sweep subsystem produces a stream of audited findings: each finding has been emitted by a nano worker, classified by category, and (if a reviewer has processed it) either accepted, rejected, or resolved. The Editorial Copilot turns that curated finding stream into structured briefing drafts — short, schema-validated summaries consumable by downstream editors or dashboards. The point of this service is *not* to be creative: it is to compose audited inputs into a constrained shape, so that every briefing a human ever reads is traceable back to an accepted finding, and never to raw model output.

2 Goals

- Accept a set of finding references and return one briefing object conforming to a strict Zod schema.
- Reject any request that would ground the briefing on unaudited content — only findings with a suitable audit state are eligible.
- Validate every model response against the output schema before returning; on failure, retry bounded times, then fail closed.
- Expose a deterministic *provenance* mapping: every field in the output cites at least one source finding id.
- Run with a bounded token budget: inputs are truncated, outputs are length-capped per field.

3 Non-Goals

Non-Goal

This spec does not cover: (a) free-form chat or conversational drafting (see SPEC-SW-012), (b) publication, scheduling, or downstream distribution of briefings, (c) multilingual translation, (d) the SEO scoring heuristic (owned by the client), (e) human-in-the-loop editing after a briefing is produced, (f) persistence of briefings beyond returning them to the caller.

4 Invariants

Invariant

No unaudited content is ever emitted. A briefing request whose finding set contains at least one finding in state `unaudited` or `rejected` is refused with a typed error; no model call is made.

Invariant

The briefing output is structured, not free-form: every response is parsed through the shared Zod schema `BriefingSchema` and rejected on failure. The service returns a typed `Briefing`, never a raw string.

Invariant

Provenance is total: for every top-level field of a returned briefing, the `provenance` map contains at least one source finding id. A briefing with orphan fields is invalid.

Invariant

All text fields are length-bounded at schema level (title $\leq$ 120 chars, summary $\leq$ 400 chars, each bullet $\leq$ 200 chars, $\leq$ 6 bullets). Overruns are rejected, not truncated silently.

Invariant

Model retries on schema-invalid output are bounded by `MAX_REPAIR_ATTEMPTS` (default 1). After the cap, the service returns a typed `BriefingError` — never a partially valid briefing.

5 Interface

Public surface; consumers include an admin briefing endpoint and internal aggregators.

```
import { z } from "zod";

export const BriefingSchema = z.object({
  title: z.string().min(8).max(120),
  summary: z.string().min(20).max(400),
  bullets: z.array(z.string().min(4).max(200)).min(1).max(6),
  tags: z.array(z.string().min(2).max(40)).max(8),
  category: z.enum([
    "trend", "anomaly", "quality", "coverage", "other",
  ]),
  // Field name -> non-empty list of source finding ids
  provenance: z.record(z.string(), z.array(z.string()).min(1)),
});

export type Briefing = z.infer<typeof BriefingSchema>;

export type BriefingError =
  | { kind: "unaudited_input"; findingIds: string[] }
  | { kind: "schema_invalid"; attempts: number; issues: string[] }
  | { kind: "model_unavailable"; cause: string };

export interface EditorialCopilot {
  draftBriefing(input: {
    findingIds: string[];
    angle?: string;      // optional user-provided framing
  }): Promise<
    | { ok: true; briefing: Briefing }
    | { ok: false; error: BriefingError }
  >;
}
```

6 Scenarios

Scenario: Nominal: audited findings produce a valid briefing

Given three finding ids all in state `accepted`
When `draftBriefing` is invoked with those ids
Then the response is `{ ok: true, briefing }`
And `briefing` validates against `BriefingSchema`
And every key of `briefing.provenance` maps to a non-empty subset of the input ids

Scenario: Refusal on unaudited input

Given one of the input finding ids is in state `unaudited`
When `draftBriefing` is invoked
Then the service returns `{ ok: false, error: { kind: "unaudited_input", findingIds: [...] } }`
And no call to the underlying model is made
And the returned `findingIds` list contains the offending id

Scenario: Schema-invalid model output is repaired once then fails closed

Given the model returns a JSON object missing the `summary` field on attempt 1
And the model returns a JSON object missing the `bullets` field on attempt 2
When `draftBriefing` is invoked
Then the service returns `{ ok: false, error: { kind: "schema_invalid", attempts: 2, issues: [...] } }`
And no partially valid briefing is emitted to the caller

Scenario: Orphan field is rejected at validation

Given the model returns a briefing with keys `title, summary, bullets, tags, category, provenance`
And `provenance` omits the `summary` key
When validation runs
Then schema validation fails with an `orphan_field` issue
And the error propagates as `schema_invalid`

Scenario: Model unavailable

Given the underlying cortex executor raises a network error
When `draftBriefing` is invoked
Then the service returns `{ ok: false, error: { kind: "model_unavailable", cause } }`
And no exception escapes the service boundary

7 Acceptance Criteria

Acceptance Criterion AC-1

`draftBriefing` returns `ok: true` with a `Briefing` that parses successfully through `BriefingSchema` when all input findings are audited-accepted.

Acceptance Criterion AC-2

`draftBriefing` returns `unaudited_input` and performs no model call when at least one input id is `unaudited` or `rejected`.

Acceptance Criterion AC-3

Every top-level field of a returned briefing appears as a key in `briefing.provenance` with a non-empty list of source ids, all drawn from the input finding ids.

Acceptance Criterion AC-4

A model response exceeding any schema length bound (e.g. `summary` > 400 chars) causes the response to be rejected; the caller sees `schema_invalid`, never a truncated briefing.

Acceptance Criterion AC-5

The number of retries on schema failure is capped at `MAX_REPAIR_ATTEMPTS`; beyond the cap the service returns `schema_invalid` with the attempt count.

Acceptance Criterion AC-6

When the cortex executor throws, `draftBriefing` returns `model_unavailable`; no exception reaches the caller.

Acceptance Criterion AC-7

The service never returns both `ok: true` and a briefing with an empty `bullets` array.

8 Traceability

AC	Test file	Test name
AC-1	`packages/sweep/tests/unit/editorial-copilot.spec.ts`	`happy path returns schema-`
AC-2	`packages/sweep/tests/unit/editorial-copilot.spec.ts`	`refuses unaudited findings`
AC-3	`packages/sweep/tests/unit/editorial-copilot.spec.ts`	`provenance covers every ou`
AC-4	`packages/sweep/tests/unit/briefing-schema.spec.ts`	`rejects overlength fields`
AC-5	`packages/sweep/tests/integration/editorial-copilot.spec.ts`	`schema repair retries are`
AC-6	`packages/sweep/tests/integration/editorial-copilot.spec.ts`	`model failure returns mode`
AC-7	`packages/sweep/tests/unit/briefing-schema.spec.ts`	`bullets non-empty invarian`

9 Open Questions

- Should `category` be inferred from the dominant finding category, or selected freely by the model within the enum?
- Do we want a soft retry on `model_unavailable` (with backoff), or leave retry policy to the caller?
- Should we expose a dry-run mode that returns only the prompt that would be sent, for audit purposes?
- How do we version `BriefingSchema`? A breaking field change affects all existing consumers.